Coinductive Symmetric Homomorphism Reinforcement Learning

A New Foundation Where Optimality Is Pure Structure

Ricardo Corral-Corral

February 2026

Abstract

We propose a new foundation for optimal sequential decision making based on *structure preservation* rather than scalar maximization. Traditional reinforcement learning defines optimality as maximizing the expected sum of discounted rewards [1]—a criterion that discards temporal structure. We formalize a stronger condition: the ranking of actions in any state must be a *coinductive symmetric homomorphism* into the ordering of infinite reward streams [5]. Concretely, if action a is ranked below action b, then at *every* future timestep $t = 0, 1, 2, \ldots$, the reward from a's trajectory must be dominated by b's.

We prove five main results, all machine-verified in Agda: **(1)** Subsumption: the coinductive property implies classical argmax optimality for any finite horizon; **(2)** Monotonic learning: ranking updates via violation-correction converge in at most $|S| \cdot \binom{|A|}{2}$ swaps; **(3)** Instant adaptation: when actions become unavailable, the next-best action is $O(1)$ lookup, not $O(|S| \cdot |A|)$ recomputation; **(4)** Stochastic extension: the framework lifts to probabilistic environments via the Giry monad, where rankings preserve *lexicographic expected* stream dominance; **(5)** Scalable verification: a modular Environment Class architecture provides automatic trace-to-stream bridges for structured domains, demonstrated with a full coinductive homomorphism for 8-Queens (8^8 search space, zero postulates). The complete pipeline—from learning to deployable policy—is validated end-to-end: materialized policy tables produce a verified 8-Queens solution by table lookup alone. This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code while producing this PDF.

1 Motivation: The Limitations of Scalar Returns

Optimal sequential decision making is conventionally defined by maximizing a scalar return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

This widely adopted criterion discards temporal structure. Consider two futures:

Future	Reward Sequence	Sum (γ=0.9)
A: "Win then die"	$[10, -5, 0, 0, \ldots]$	$10 - 4.5 = 5.5$
B: "Struggle then thrive"	$[0, 5, 0, 0, \ldots]$	$0 + 4.5 = 4.5$

A scalar optimizer chooses Future A. But *structurally,* Future B may be preferable—it avoids catastrophic failure. The sum erases this distinction. This is **value aliasing**: structurally different futures mapping to the same scalar.

Several well-known artifacts follow from scalar aggregation:

- **The Discount Factor (γ):** An extrinsic parameter that controls how quickly future rewards are attenuated.

- **Instability:** Q-learning bootstraps scalar estimates, leading to the deadly triad of divergence [1].
- **Credit Assignment:** A reward at $t = 1000$ is discounted by $\gamma^{1000} \approx 0$, making long-horizon learning difficult.

We propose to define optimality via *structure preservation* rather than scalar maximization.

2 Structural Optimality: The Key Idea

Consider a maze with five actions: up, down, left, right, wait. Some lead to reward, others to penalty, one is blocked. Traditional RL assigns a scalar value to each action and selects the maximum.

We propose a different criterion:

> **An agent is optimal when the ranking of actions mirrors the ordering of their infinite futures—pointwise, at every timestep.**

When this structure-preservation property holds, the agent is optimal in a strong sense. When it does not, learning amounts to *restoring the correspondence* between rankings and outcomes.

3 Formal Definition: Coinductive Symmetric Homomorphism

We now formalize this intuition. The coinductive symmetric homomorphism has three components:

3.1 The Domain: Actions and Rankings

Let $\mathcal{A}$ be the finite set of actions. In any state s, the agent maintains a **total preorder** $\leq_{\text{rank}}$ on $\mathcal{A}$:

$$a \leq_{\text{rank}} b \quad \Longleftrightarrow \quad \text{“action } b \text{ is at least as good as action } a\text{”}$$

This is what the agent *believes*. When is this belief *correct*?

3.2 The Codomain: Infinite Outcome Streams

Let $\mathcal{O}$ be the set of infinite outcome streams. We equip $\mathcal{O}$ with a **coinductive dominance order**:

$$x \leq_s y \iff \text{head}(x) \leq_r \text{head}(y) \wedge \text{tail}(x) \leq_s \text{tail}(y)$$

Remark 1. This is *coinductive*: the definition refers to itself on the tails. Unlike induction (which builds up from base cases), coinduction unfolds forever. Stream x dominates y if and only if **at every time step** $t = 0, 1, 2, \ldots$, the t-th element of x is $\leq_r$ the t-th element of y.

3.3 The Homomorphism: Structure Preservation

Let $h : \mathcal{A} \to \mathcal{O}$ map each action to its infinite future stream.

Definition 1 (Coinductive Symmetric Homomorphism)**.** The map h is a **coinductive symmetric homomorphism** if:

$$a \leq_{\text{rank}} b \implies h(a) \leq_s h(b)$$

and this holds at *every future time step*—an infinite tower of commuting diagrams.

Remark 2 (On "Symmetric"). The term "symmetric" reflects two aspects: (1) action rankings are permutations—elements of the symmetric group $S_{|A|}$; (2) the correspondence between coinductive stream ordering and pointwise dominance is an *isomorphism*: $x \leq_s y \Leftrightarrow \forall n.\, x_n \leq_r y_n$. This bidirectional correspondence is proven in §7 and justifies treating the homomorphism as a true structure-preserving map.

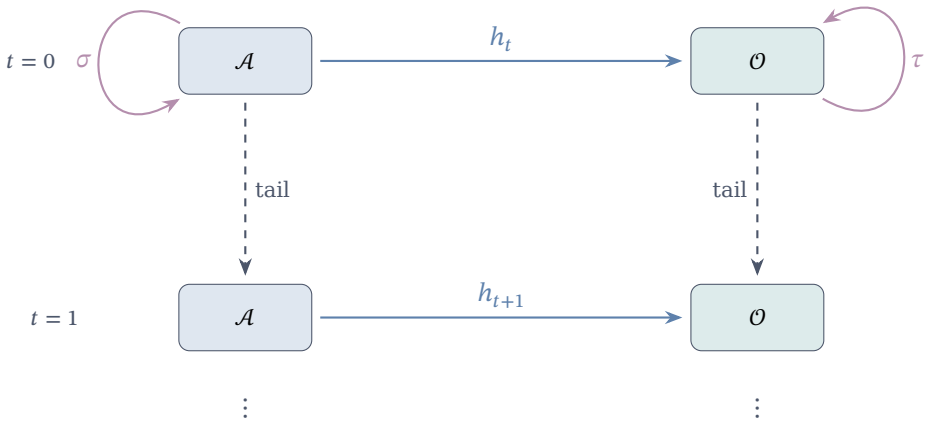

Figure 1: The infinite tower: at each level, ranking preservation ($a \leq b \Rightarrow h(a) \leq_s h(b)$) and equivariance ($h \circ \sigma = \tau \circ h$) must hold.

An agent is optimal if and only if its ranking of actions mirrors the dominance of their infinite futures—not just at the current time step, but at every future moment. When this correspondence is violated, learning amounts to restoring it.

4 The Agda Core: Verified Implementation

We now present the verified implementation. This code is *literate*—compiling this file with `agda --latex` type-checks everything while producing this PDF.

```
{-# OPTIONS --safe --guardedness #-}

module CSHRL where

open import Data.List using (List; map; foldr; []; _∷_)
open import Codata.Guarded.Stream using (Stream; head; tail; _∷_; tabulate)
open import Relation.Binary.PropositionalEquality using (_≡_; _≢_; refl)
open import Data.Product using (proj₁; proj₂; _×_; _,_)
open import Relation.Nullary using (¬_; Dec; yes; no)
open import Function using (_∘_)
open import Data.Nat using (ℕ; zero; suc; _⊔_)
open import Data.Bool using (Bool; true; false; if_then_else_)
```

The Core module is parameterized by the environment:

```
module Core
  (State Action Reward : Set)
  (step       : State → Action → State × Reward) -- Transition function
  (_≤ᵣ_       : Reward → Reward → Set)           -- Reward ordering (propositional)
  (max        : Reward → Reward → Reward)        -- Max for computing supremum
  (bottom     : Reward)                          -- Minimum reward
  (all-actions : List Action)                    -- Finite action space
  where
```

```
infix 4 _≤ₛ_

-- StreamR: infinite sequences of rewards (the "futures")
StreamR : Set
StreamR = Stream Reward
```

The module is parameterized over states, actions, and rewards. The `step` function defines the environment dynamics; rewards carry a propositional ordering `_≤ᵣ_` (as opposed to a Boolean decision procedure—see §4.4).

```
-- Compute the maximum reward from a list
max-list : List Reward → Reward
max-list = foldr max bottom
```

4.1 Value and Action-Value Streams

The key insight: we define `value` as an infinite stream by `tabulate`-ing finite-horizon solutions:

```
-- Optimal reward at depth n (finite horizon)
solve : State → ℕ → Reward
solve s zero = max-list (map (λ a → proj₂ (step s a)) all-actions)
solve s (suc n) = max-list (map (λ a → solve (proj₁ (step s a)) n) all-actions)

-- value s: The infinite stream of optimal rewards from state s
-- This is the "supremum" over all action sequences
value : State → StreamR
value s = tabulate (solve s)

-- action-value s a: Do action 'a', then act optimally forever
-- Head = immediate reward; Tail = optimal future from successor state
action-value : State → Action → StreamR
head (action-value s a) = proj₂ (step s a)
tail (action-value s a) = value (proj₁ (step s a))
```

The `action-value` of taking action a in state s is an infinite stream whose head is the immediate reward and whose tail is the optimal stream from the successor state. This mirrors the Bellman decomposition without collapsing to a scalar.

4.2 Coinductive Stream Ordering

The central ordering compares infinite futures coinductively:

```
-- The coinductive ordering on streams
-- x ≤ₛ y means: at EVERY time step, head x ≤ᵣ head y, forever
record _≤ₛ_ (x y : StreamR) : Set where
  coinductive -- coinductive: the definition unfolds to arbitrary depth
  field
    head≤ : head x ≤ᵣ head y -- Now: first rewards compare
    tail≤ : tail x ≤ₛ tail y      -- Forever: tails recursively compare

open _≤ₛ_ public
```

The `coinductive` keyword instructs Agda that this record may be infinitely deep. A proof of `x ≤ₛ y` is therefore an infinite witness: at every time step, the heads compare correctly.

4.3 The Symmetric Homomorphism

The central definition packages a ranking with its preservation proof:

```
-- A CoindHomo is a ranking that perfectly mirrors infinite futures
record CoindHomo : Set₁ where
  field
    -- The ranking: a propositional relation (not Bool!)
    _≤ₐ_ : State → Action → Action → Set

    -- THE KEY PROPERTY: ranking mirrors action-values
    -- If a ≤ₐ b, then the future of a is dominated by the future of b
    preserves : ∀ a b s → _≤ₐ_ s a b →
                action-value s a ≤ₛ action-value s b

open CoindHomo {{...}} public
```

A `CoindHomo` witness pairs a ranking relation `_≤ₐ_` with a proof that it preserves into the coinductive ordering: whenever the ranking asserts $a \leq b$, the infinite future of a is dominated by that of b, verified statically by Agda's type checker.

4.4 Why Propositions Instead of Booleans?

- **Booleans are blind:** `true` carries no information about *why*. To use it, we need a separate soundness lemma.
- **Propositions carry evidence:** A value of type `r₁ ≤ᵣ r₂` *is* the proof.
- **Decidability bridges both:** `Dec (r₁ ≤ᵣ r₂)` is either `yes p` (with proof) or `no ¬p` (with refutation).

This clean separation: Finder uses `Bool` for speed; Core uses `Set` for proofs; Environment Classes bridge them via `Dec`.

5 Scope, Assumptions, and Limits

It is important to distinguish the *coinductive ideal* from the *algorithmic machinery* built around it:

- **Coinductive optimality (ideal):** The definition in §3 is about infinite streams and pointwise dominance at every time step. This is the mathematical target.
- **Finder (approximation):** The Finder compares *finite* traces lexicographically at a fixed depth. It is a computational proxy that becomes exact under additional conditions verified by the Environment Class infrastructure (e.g., horizon sufficiency for placement problems; see §10.6).
- **Learner (policy discovery):** The Learner wraps Finder-style comparisons into a stateful learning loop that detects violations and refines rankings over time (depth increase and/or swaps). It is an algorithmic policy finder, not part of the coinductive definition itself.

The deterministic framework (Sections 2–9) assumes:

- **Finite action space:** All actions must be listed explicitly (`all-actions`).
- **Deterministic step function:** `step : State → Action → State × Reward`.
- **Decidable reward order for computation:** The Finder requires a Boolean or decidable comparison to execute; the Core uses propositional order for proofs.
- **Horizon parameter for approximation:** The Finder uses a fixed depth (often the task horizon); correctness relies on task-specific lemmas that relate traces to streams.

- **Preservation is discharged by ECs:** The proof that a ranking preserves the coinductive order is provided by the Environment Class infrastructure. For the general EC, the task supplies a direct preservation proof; for specialised ECs such as CombinatorialPlacementMDP, the bridge is constructed automatically from domain-specific lemmas (§10.3).

Section 11 extends the framework to **stochastic environments** via the Giry monad, where step functions return probability distributions. This extension is self-contained—all necessary concepts are introduced there.

6 CSHRL Subsumes Classical RL

A natural question is whether this stronger definition recovers classical results. It does.

Theorem 1 (CSHRL Subsumes Argmax)**.** *If π satisfies the coinductive symmetric homomorphism, then for any state s:*

$$\pi(s) = \arg\max_a \sum_{t=0}^{N} \gamma^t r_t(s,a) \quad \textit{for all finite } N$$

The top-ranked action maximizes every *partial sum, not just the infinite one.*

The key insight is that pointwise stream dominance implies partial-sum dominance. We formalize this with verified Agda code (presented here within Core for expository flow; in the source tree these definitions live in `appendix/Arithmetic.agda`):

```
-- Extract the n-th element of a reward stream
iter-head : ℕ → StreamR → Reward
iter-head zero s = head s
iter-head (suc n) s = iter-head n (tail s)

-- Pointwise dominance: at every index, xₙ ≤ᵣ yₙ
PointwiseDominance : StreamR → StreamR → Set
PointwiseDominance x y = ∀ n → iter-head n x ≤ᵣ iter-head n y

-- KEY LEMMA: coinductive ≤ₛ implies pointwise dominance
-- This is the bridge from infinite structure to finite sums
≤ₛ-to-pointwise : ∀ {x y} → x ≤ₛ y → PointwiseDominance x y
≤ₛ-to-pointwise p zero = head≤ p
≤ₛ-to-pointwise p (suc n) = ≤ₛ-to-pointwise (tail≤ p) n
```

Theorem 2 (Subsumption of Classical Argmax)**.** *If a ranking satisfies the `CoindHomo` property, then for any finite horizon N: if $a \leq_a b$ in the ranking, then $\sum_{t=0}^{N-1} r_t^a \leq_r \sum_{t=0}^{N-1} r_t^b$.*

The proof extends Core with reward arithmetic (addition and its monotonicity). The full verified implementation is in `src/appendix/Arithmetic.agda`; here we show the key components:

```
-- Partial sum of first N elements
partial-sum : ℕ → StreamR → Reward
partial-sum zero _ = bottom
partial-sum (suc n) s = head s +ᵣ partial-sum n (tail s)

-- Partial sum respects pointwise dominance (by induction on N)
partial-sum-mono : ∀ N x y → PointwiseDominance x y →
                   partial-sum N x ≤ᵣ partial-sum N y
partial-sum-mono zero _ _ _ = ≤ᵣ-refl
partial-sum-mono (suc n) x y pw =
  +ᵣ-mono-≤ (pw zero) (partial-sum-mono n (tail x) (tail y) (pointwise-tail pw))
```

```
-- SUBSUMPTION: preservation + ≤ₛ-to-pointwise + partial-sum-mono
subsumes-partial-sum : (homo : CoindHomo) → ∀ s a b N →
  CoindHomo._≤ₐ_ homo s a b →
  partial-sum N (action-value s a) ≤ᵣ partial-sum N (action-value s b)
subsumes-partial-sum homo s a b N p =
  partial-sum-mono N _ _ (≤ₛ-to-pointwise (CoindHomo.preserves homo a b s p))
```

The coinductive property is strictly stronger than classical argmax optimality—but when it holds, the latter follows as a corollary. A ranking satisfying `CoindHomo` automatically identifies the argmax for any finite horizon.

Stochastic extension: This subsumption result lifts to stochastic MDPs. By linearity of expectation, comparing partial sums of expected rewards is equivalent to comparing expected partial sums—exactly what classical RL optimizes. Section 11 presents the stochastic framework; the Appendix provides the complete proof (`appendix/StochasticSubsumption.agda`).

7 The Stream Isomorphism

We defined stream dominance coinductively: $x \leq_s y$ iff heads compare and tails recursively compare. A natural question is whether this definition captures exactly pointwise dominance at every timestep. The answer is affirmative: the coinductive definition is *isomorphic* to pointwise dominance.

```
-- THE REVERSE DIRECTION: pointwise → coinductive
pointwise-to-≤ₛ : ∀ {x y} → PointwiseDominance x y → x ≤ₛ y
head≤ (pointwise-to-≤ₛ pw) = pw zero
tail≤ (pointwise-to-≤ₛ pw) = pointwise-to-≤ₛ (pointwise-tail pw)
```

Together with `≤ₛ-to-pointwise`, this yields:

$$x \leq_s y \iff \forall n.\, x_n \leq_r y_n$$

This isomorphism has three consequences. First, it confirms that the coinductive formulation exactly captures "dominance at every timestep"—it is a precise characterization, not an approximation. Second, it establishes Finder completeness: since the Finder compares traces at finite depths, the isomorphism guarantees that trace dominance at all depths implies coinductive stream dominance. Third, it justifies the "symmetric" in our title: the bidirectional correspondence means that rankings preserving stream order and stream orders inducing rankings are interchangeable views.
The full proof is in `appendix/StreamIsomorphism.agda` and the Appendix document.

8 Related Work and Positioning

CSHRL connects to multiple research traditions but makes a distinct move: replacing scalar aggregation with a coinductive order on infinite futures. We situate our contribution relative to classical RL, lexicographic multi-objective RL, constrained/safe RL, and coalgebraic foundations.

8.1 Classical Value-Based RL

Approach	Core idea	CSHRL relationship
Classical RL [1]	Optimize $\sum_t \gamma^t r_t$ via Bellman equations	CSHRL *subsumes* this: if the coinductive property holds, argmax follows (§6)
Policy Gradients [2]	Gradient ascent on expected return	Optimization *method*; CSHRL defines the *target*
Max-Entropy RL [3]	Add entropy bonus to encourage exploration	Scalar returns with exploration add-on; CSHRL's full-ranking objective dissolves exploration/exploitation (Remark 3)
AIXI [4]	Universal prior over computable environments	Uncomputable ideal; CSHRL is also an ideal but *structurally* defined
Coalgebra [5]	Infinite behaviors via coinductive definitions	Direct inspiration: $\preceq_s$ is a coinductive relation in Rutten's sense
MDP Symmetries [6]	Exploit state/action symmetries for efficiency	Complementary: CSHRL uses symmetry as *optimality criterion*, not just speedup
Giry Monad [10]	Probability distributions as a monad on measurable spaces	Direct application: stochastic CSHRL uses discrete Giry monad (§11)

8.2 Lexicographic Multi-Objective RL

The most closely related line of work is **lexicographic multi-objective RL** (LMORL), which optimizes multiple reward signals according to strict priority: first maximize the highest-priority objective, then the next, and so on.

Skalse et al. [11] introduce both value-based and policy-gradient algorithms that provably converge to lexicographically optimal policies. Their framework treats objectives as separate reward functions ordered by importance. Tercan and Prabhu [12] address limitations of earlier heuristics—particularly cases where Bellman equations fail to respect lexicographic constraints—via the *Lexicographic Projection Algorithm*.

Key distinction: LMORL orders *objectives* (different reward signals) lexicographically. CSHRL orders *timesteps*: the same reward signal is compared pointwise at $t = 0$, then $t = 1$, then $t = 2$, forever. This is a fundamentally different structure:

- LMORL: "Safety reward first, then efficiency reward."
- CSHRL: "At every moment t, the better action's reward dominates."

The coinductive order $\preceq_s$ in CSHRL is not a priority over objectives but a *temporal* dominance relation. When two actions yield the same reward at $t = 0$, we compare $t = 1$; if equal again, we compare $t = 2$; and so on *ad infinitum*. This is closer to lexicographic comparison of infinite sequences (as in ordinal arithmetic) than to multi-objective scalarization.

8.3 Constrained and Safe RL

Safe RL [13] treats some objectives as hard constraints rather than soft trade-offs. Constrained MDPs optimize a primary reward while ensuring constraint satisfaction (e.g., collision probability $< \epsilon$).

CSHRL's relationship to safe RL is complementary:

- **Temporal constraints as dominance:** A safety constraint "never enter state X" translates to: the safe action's trace dominates at every timestep (the unsafe action's trace eventually shows the penalty).
- **No constraint relaxation:** CSHRL's coinductive order is strict—there is no ϵ-violation. If a's trace dominates b's at every step, the dominance is absolute.

8.4 Preference-Based, Ordinal, and Ranking RL

Preference-based RL (PbRL) [14] learns from pairwise comparisons rather than scalar rewards. This connects to CSHRL's ranking structure, but the relationship is nuanced:

- PbRL *infers a latent scalar* from comparisons, then optimizes that scalar. CSHRL requires only *ordinal structure*—a partial order on outcomes—and compares streams directly without scalar collapse.
- CSHRL's Core module is parameterized by `_≤ᵣ_ : Reward → Reward → Set`—any partial order suffices. Human preference orderings are a valid instantiation.

Direct Preference Optimization (DPO) [18] is a prominent recent instance of this pattern: it bypasses explicit reward modelling by deriving an implicit scalar reward from pairwise preferences via the Bradley-Terry model, then optimizes a policy to maximize it. The pairwise structure of the input is discarded once the implicit reward is extracted. In CSHRL, by contrast, the pairwise ordering *is* the optimality criterion—preserved coinductively, not collapsed into a scalar.

Deep Ordinal Reinforcement Learning [15] modifies Q-learning for ordinal rewards via thresholded updates, mitigating overestimation in DQN. However, it retains discounts and bootstrapping, unlike CSHRL's coinductive elimination of γ.

Recent extensions in RLHF, such as Ordinal Probabilistic Reward Models [16], treat rewards as ordinal distributions for human feedback alignment. These are empirical and lack CSHRL's formal guarantees (monotonic convergence, $O(1)$ adaptation).

8.5 Coalgebraic Foundations

The coinductive stream order $\leq_s$ is a coalgebraic relation in the sense of Rutten [5]. Coalgebra provides the categorical semantics for infinite behaviors, and our formalization directly instantiates these ideas in Agda's guarded coinduction. The `coinductive` keyword in the $\leq_s$ record is precisely the coalgebraic final coalgebra construction.

MDP homomorphisms [6] exploit state/action symmetries for computational efficiency. CSHRL uses a different notion of "symmetry": not group-theoretic automorphisms of the state space, but *order-preservation* between rankings and stream dominance.

8.6 Formal Verification of RL

CertRL [17] is a Coq library that machine-checks convergence proofs for value and policy iteration on finite MDPs, using the Giry monad for probabilistic reasoning. While CertRL provides rigorous guarantees for classical tabular RL, CSHRL extends formal verification to a different paradigm: coinductive structure preservation rather than scalar convergence. Both projects demonstrate that machine-verified RL theory is feasible; they target complementary foundations.

8.7 What Distinguishes CSHRL

- **Criterion vs. algorithm:** Classical RL and policy gradients are *algorithms* to optimize a scalar. CSHRL is an *optimality criterion*—a structural property that rankings may or may not satisfy.
- **Temporal structure:** The scalar return $\sum \gamma^t r_t$ erases timing; the coinductive order $\leq_s$ preserves it completely.
- **Pointwise dominance:** Unlike lexicographic MORL (priority among objectives) or average-reward RL (limit of partial sums), CSHRL requires dominance at *every* timestep.
- **Machine verification:** Unlike most RL theory, CSHRL is fully implemented in Agda with type-checked proofs. CertRL [17] is the closest related effort, targeting classical RL in Coq.

9 The Algorithm: Violation-Correction Search

Given the coinductive definition of optimality, the question is how to *compute* an optimal ranking. The Finder algorithm uses lexicographic trace comparison and requires no discount factor.

```
module Finder
  (State Action Reward : Set)
  (step           : State → Action → State × Reward)
  (_≤?_           : Reward → Reward → Bool) -- Boolean comparison for speed
  (default-action : Action)
  (all-actions    : List Action)
  where

  open import Data.List using (List; map)

  -- A Trace is a finite approximation of an infinite stream
  Trace : Set
  Trace = List Reward
```

9.1 Lexicographic Comparison

Compare traces element-by-element—the first difference decides:

```
-- Lexicographic ordering: compare head-to-head until one wins
_≤ₜ_ : Trace → Trace → Bool
[]     ≤ₜ []     = true
[]     ≤ₜ (_ ∷ _) = true  -- Shorter is less (or equal prefix)
(_ ∷ _) ≤ₜ []     = false -- Longer is greater
(r₁ ∷ t₁) ≤ₜ (r₂ ∷ t₂) =
  if r₁ ≤? r₂ then
    if r₂ ≤? r₁ then (t₁ ≤ₜ t₂) -- Equal heads: recurse on tails
    else true -- r₁ < r₂: strictly better
  else false  -- r₁ > r₂: strictly worse
```

Unlike summation (which discards timing information), lexicographic comparison preserves it: a trace $[10, -5]$ dominates $[0, 5]$ because $10 > 0$ at step 0, regardless of subsequent elements.

9.2 Trace Evaluation

Compute the best trace from any state:

```
mutual
  best-trace : State → ℕ → Trace
  best-trace s zero = []
  best-trace s (suc k) =
    let outcomes = map (λ a → trace-action s a k) all-actions
    in max-trace outcomes

  trace-action : State → Action → ℕ → Trace
  trace-action s a k =
    let (s' , r) = step s a
        future = best-trace s' k
    in r ∷ future

  max-trace : List Trace → Trace
  max-trace [] = []
  max-trace (t ∷ ts) = max-helper t ts
```

```
max-helper : Trace → List Trace → Trace
max-helper current [] = current
max-helper current (t ∷ ts) =
  if current ≤ₜ t
  then max-helper t ts
  else max-helper current ts
```

9.3 The Ranking Finder

Sort actions by their traces to get a full ranking:

```
insert : (Action × Trace) → List (Action × Trace) → List (Action × Trace)
insert x [] = x ∷ []
insert (a₁ , t₁) ((a₂ , t₂) ∷ xs) =
  if t₂ ≤ₜ t₁
  then (a₁ , t₁) ∷ (a₂ , t₂) ∷ xs
  else (a₂ , t₂) ∷ insert (a₁ , t₁) xs

sort-scored : List (Action × Trace) → List (Action × Trace)
sort-scored [] = []
sort-scored (x ∷ xs) = insert x (sort-scored xs)

find-ranking : State → ℕ → List Action
find-ranking s k =
  let scored = map (λ a → (a , trace-action s a k)) all-actions
      sorted = sort-scored scored
  in map proj₁ sorted

find-policy : State → ℕ → Action
find-policy s k =
  let ranking = find-ranking s k
  in head-default ranking
  where
    head-default : List Action → Action
    head-default [] = default-action
    head-default (x ∷ _) = x
```

Note that no discount factor is required. Standard RL needs $\gamma < 1$ to ensure convergence of the sum; here we compare structure directly, and the lexicographic order is well-defined at any depth.

10 Environment Classes: Modularity Without Postulates

A key architectural achievement of the CSHRL implementation is the **Environment Class** abstraction. Rather than proving preservation from scratch for each task, we factor out common structure into reusable modules.

10.1 The Problem: Boilerplate and Trust

Without environment classes, every task would need:

1. Its own Finder algorithm implementation
2. Its own trace-to-stream bridge lemma
3. Its own preservation proof skeleton

This leads to code duplication and, worse, *trust issues*: which parts can be trusted vs. which need re-verification?

10.2 The Solution: Parameterized Modules

Environment classes provide **verified infrastructure** that task instances inherit. The full implementation is in `src/CSHRL/EnvironmentClass/`; here we show the module signature:

```
-- The FiniteDeterministicMDP environment class signature
record ECParameters : Set₁ where
  field
    State Action Reward : Set
    step     : State → Action → State × Reward
    _≤ᵣ_     : Reward → Reward → Set
    _≤?_     : (r s : Reward) → Dec (r ≤ᵣ s) -- Decidable!
    ≤ᵣ-refl  : ∀ {r} → r ≤ᵣ r
    max      : Reward → Reward → Reward
    bottom   : Reward
    all-actions : List Action
    default-action : Action
    horizon : ℕ


-- AUTOMATICALLY PROVIDED by the EC:
-- • Trace type and orderings (_≤ₜ_, _≤ₜᵇ_)
-- • Finder algorithm (best-trace, find-ranking, find-policy)
-- • Ranking relation (_ranks_≤_)
-- • Trace-to-stream bridge infrastructure (trace-≤ₜ-head)
-- • Stream reflexivity (≤ₛ-refl)
-- • Full CoindHomo (for specialised ECs like CombinatorialPlacementMDP)
```

10.3 What Task Instances Must Provide

The burden on a task instance depends on which environment class it instantiates.

FiniteDeterministicMDP (general case). A task provides:

1. **Domain specifics:** The `State`, `Action`, `step` function.
2. **A direct preservation proof:** A proof that the Finder's ranking preserves action-value ordering—the full bridge from finite traces to infinite streams.

The preservation proof type has this shape:

```
-- What a task instance must prove (simplified signature)
-- Full version in src/CSHRL/EnvironmentClass/FiniteDeterministicMDP.agda
DirectPreservesType : Set → Set → Set → Set₁
DirectPreservesType State Action Reward =
  (s : State) → (a b : Action) →
  Set -- "s ranks a ≤ b" → "action-value s a ≤ₛ action-value s b"
```

CombinatorialPlacementMDP (placement problems). The specialised EC provides the trace-to-stream bridge *automatically*. A task needs only:

1. **Domain specifics:** `Config`, `Action`, `place`, constraint predicates (`is-dead`, `is-solved`).
2. **Absorbing-state proofs:** That dead states always solve to 0 and solved states always solve to the solved reward, for every depth n—typically one-line inductions.
3. **Horizon sufficiency** (`solve-horizon-suf`): If `solve` applied to an ongoing configuration at the game horizon yields 0, then it yields 0 at *every* depth. Intuitively: if a partial configuration is unsolvable within the horizon, it is unsolvable at any depth.

From these three ingredients the EC's `WithTraceBridge` module constructs a full `CoindHomo`—the complete proof that the Finder's ranking mirrors the coinductive ordering of infinite reward streams. No additional bridge reasoning is required from the task author; see §10.6 for the 8-Queens validation.

10.4 Why This Matters: Postulate-Free Verification

The environment class architecture means:

- **Verified tasks** (like TwoState, Queens1, and Queens8) have **no postulates**—every claim is machine-checked. In particular, Queens8 achieves full coinductive homomorphism verification for a non-trivial combinatorial problem ($N = 8$), demonstrating that the EC architecture scales beyond toy examples.
- **Classic tasks** (like Maze, KeyDoor) can use postulates for properties that are difficult to formalise, but the *structural* preservation proof is still verified.
- **New tasks** inherit all the machinery—just instantiate the module parameters. For placement problems, the trace-to-stream bridge is entirely automatic (§10.3).

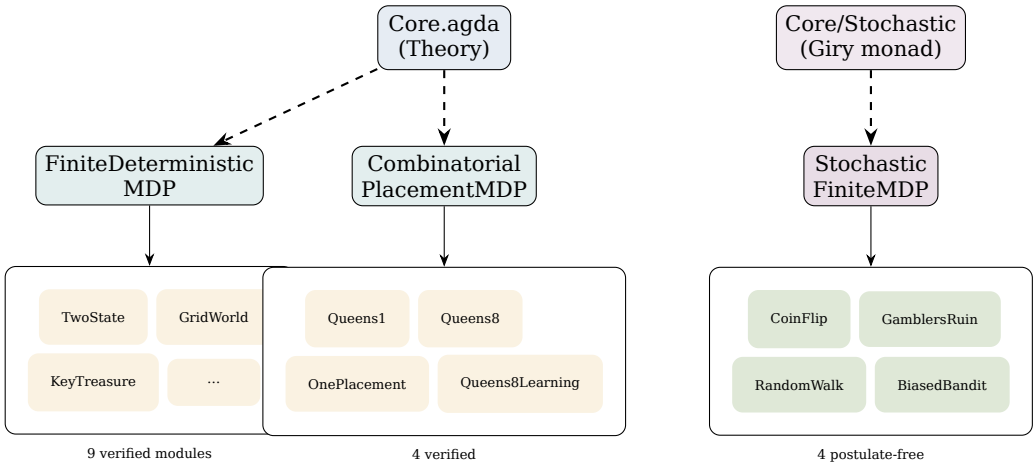

Figure 2: Environment class hierarchy with representative tasks. Core provides theory; ECs provide reusable verification infrastructure; task groups contain domain-specific instantiations. The full catalogue comprises 23 modules: 9 deterministic verified (TwoState, GridWorld, KeyTreasure, DelayedGrat, plus 5 learning/test modules), 4 combinatorial placement (OnePlacement, Queens1, Queens8, Queens8Learning), 4 stochastic (CoinFlip, GamblersRuin, RandomWalk, BiasedBandit), and 6 classic demos without EC (Maze, KeyDoor, Queens, Trap, Energy, DelayedGrat).

10.5 FiniteDeterministicMDP: The General Case

The base environment class for finite deterministic MDPs. Tasks like TwoState, GridWorld, KeyTreasure, and DelayedGratification instantiate this EC. The key components:

- **Parameters:** State, Action, Reward types; step function; reward ordering; action list
- **Provided:** Value streams, action-value streams, lexicographic trace comparison, Finder integration
- **Required from task:** Direct preservation proof (see §10.3 for what each EC requires)

The full implementation is in `src/CSHRL/EnvironmentClass/FiniteDeterministicMDP.agda`. Most tasks inherit directly from this EC.

10.6 CombinatorialPlacementMDP: Constraint Satisfaction

A *specialized* environment class for placement problems (N-Queens, Sudoku, Graph Coloring), extending FiniteDeterministicMDP with domain-specific structure. The full implementation is in `src/CSHRL/EnvironmentClass/CombinatorialPlacementMDP.agda`:

```
-- Placement EC parameters
record PlacementParameters : Set₁ where
  field
    Config : Set                    -- Partial/complete configurations
    Action : Set                    -- Placement actions
    is-dead : Config → Bool         -- Constraint violated?
    is-solved : Config → Bool       -- Complete valid solution?
    place : Config → Action → Config -- Apply placement
    solved-reward : ℕ               -- Reward for solutions
    all-actions : List Action
    default-action : Action         -- Fallback action
    horizon : ℕ                     -- Max placements needed

-- States are automatically classified by the EC:
data PlacementState (Config : Set) : Set where
  Ongoing : Config → PlacementState Config -- Still placing
  Dead    : PlacementState Config          -- Violated (absorbing, reward 0)
  Solved : Config → PlacementState Config  -- Solution (absorbing, reward R)
```

The key structural property is that Dead states produce stream $[0, 0, 0, \ldots]$ and Solved states produce $[R, R, R, \ldots]$. Any path reaching Solved therefore dominates any path reaching Dead at every timestep—precisely the condition required by the coinductive order.

Internal module architecture. The EC factors the preservation proof into reusable layers, each parameterised by progressively stronger assumptions:

1. `WithAbsorbingLemmas` (parameters: `solve-Dead-0`, `solve-Solved-R`). Derives stream-level domination lemmas for absorbing states (e.g., `Dead` $\leq_s s$ for all s).
2. `WithBinaryStructure` (adds: `0<R`). Proves that `solve` is *binary*—every value is either 0 or R—and derives monotonicity (`solve-mono`: once a solve value reaches R, it stays R at greater depth). These lemmas hold generically for all placement problems and do *not* depend on `solve-horizon-suf`, avoiding a circular dependency.
3. `WithTraceBridge` (adds: `solve-horizon-suf`). Combines binary structure with horizon sufficiency to construct the full `CoindHomo`: the Finder's trace-based ranking `_ranks_≤_` preserves the coinductive ordering `_≤ₛ_` of action-value streams.

This layered design means the task author supplies at most four simple ingredients (the placement parameters, two absorbing-state lemmas, and horizon sufficiency), while the EC machinery handles the entire bridge from finite traces to infinite streams.

Validation: 8-Queens ($N = 8$). The `Queens8` module (`src/CSHRL/Tasks/Verified/Queens8.agda`) instantiates the EC for the 8-Queens problem with `--safe` and zero postulates. The central effort is the proof of `solve-horizon-suf`: if a partial board configuration cannot reach a solution within $N = 8$ placement steps, then it cannot reach a solution at *any* depth. The argument proceeds by well-founded induction on $N - |config|$ (the number of remaining placement slots):

- **Base case** ($|config| \geq N$): After N placements the board is full; `is-solved` must have fired or the configuration is dead.

- **Inductive step**: The binary structure of `solve` lets us decompose a zero-valued ongoing state into the conjunction that *every* successor has solve-value 0, via a symbolic argument (`solve-component-0`) that avoids pattern matching on stuck terms. Each successor is classified as Dead (trivially zero), Solved (contradiction), or Ongoing (inductive hypothesis with reduced fuel).

With `solve-horizon-suf` discharged, opening `WithTraceBridge` yields a complete `CoindHomo`: for every state s and actions a, b, if the Finder ranks $a \leq b$, then the infinite reward stream of a is dominated by that of b at every time step. This is the strongest form of correctness in the framework—it implies classical finite-horizon optimality as a corollary (by the Subsumption theorem of §6)—and is achieved for a combinatorial search space ($8^8 = 16{,}777{,}216$ candidate placements) where direct stream enumeration is infeasible.

11 Stochastic Extension via the Giry Monad

The deterministic framework extends naturally to probabilistic environments. We implement this via a **finite distribution monad**—a discrete approximation of the Giry monad [10].

11.1 The Distribution Monad

The core idea: replace deterministic outcomes with weighted distributions over outcomes.

```
-- Finite distribution: list of (outcome, weight) pairs
Dist : Set → Set
Dist A = List (A × ℕ)

-- Pure: deterministic distribution (single outcome with weight 1)
pure : ∀ {A} → A → Dist A
pure a = (a , 1) ∷ []

-- Scale all weights by a factor
scale : ∀ {A} → ℕ → Dist A → Dist A
scale k = map (λ { (a , w) → a , k * w })

-- Bind: compose distributions (Kleisli extension)
_»=_ : ∀ {A B} → Dist A → (A → Dist B) → Dist B
d »= f = concatMap (λ { (a , w) → scale w (f a) }) d
```

The full implementation is in `src/CSHRL/Probability/Finite.agda`.

11.2 Stochastic Step Functions

The key change: the step function returns a distribution over outcomes instead of a single outcome.
Example (CoinFlip): A fair coin flip:

```
-- CoinFlip MDP states and actions
data CoinState : Set where
  Ready Won Lost : CoinState

data CoinAction : Set where
  Flip Stay : CoinAction

-- Bernoulli distribution helper
bernoulli : ∀ {A} → A → ℕ → A → ℕ → Dist A
bernoulli a₁ w₁ a₂ w₂ = (a₁ , w₁) ∷ (a₂ , w₂) ∷ []

-- Stochastic step function
```

```
coin-step : CoinState → CoinAction → Dist (CoinState × ℕ)
coin-step Ready Flip = bernoulli (Won , 1) 1 (Lost , 0) 1 -- 50% Win, 50% Lose
coin-step Ready Stay = pure (Ready , 0) -- Deterministic stay
coin-step Won _ = pure (Won , 0)        -- Terminal
coin-step Lost _ = pure (Lost , 0)      -- Terminal
```

11.3 Lexicographic Coinductive Comparison

The deterministic framework uses *pointwise* stream dominance: at every timestep t, $r_t(a) \leq r_t(b)$. For stochastic MDPs with expected values, this is **overly restrictive**—it requires dominance at *every* future step even when early rewards already determine the outcome.

The correct notion is **lexicographic coinductive comparison**: compare expected heads; if they differ, the comparison is decided. Only recurse to tails when heads are *equal*:

```
-- Lexicographic stream dominance (coinductive)
-- Key difference: tail≤ is CONDITIONAL on heads being equal
record _≤ₛ-lex_ {Reward : Set} (_≤ᵣ_ : Reward → Reward → Set)
               (x y : Stream Reward) : Set where
  coinductive
  field
    head≤ : head x ≤ᵣ head y
    tail≤ : head x ≡ head y → _≤ₛ-lex_ _≤ᵣ_ (tail x) (tail y)
```

This is **still coinductive**—the infinite tower structure is preserved—but captures the correct semantics: earlier rewards break ties, and only equal heads require recursive comparison.

11.4 Why Lexicographic Works

Consider the CoinFlip example at state `Ready`:

- `Stay`: expected head = 0
- `Flip`: expected head = 1 (unnormalized)

For the ranking `Stay` ≤ `Flip`:

- `head≤`: $0 \leq 1$ □
- `tail≤`: requires $0 \equiv 1$, which is **impossible**!

Since the condition $0 \equiv 1$ has no inhabitants, the tail proof is trivially satisfied via absurd pattern matching:

```
-- The impossible case: absurd pattern for 0 ≡ 1
coinflip-tail-example : ∀ {A : Set} → 0 ≡ 1 → A
coinflip-tail-example () -- Empty pattern: no constructor for 0 ≡ 1
```

This makes CoinFlip **fully verified with no postulates**.

11.5 The Proof Pattern

All four verified stochastic examples (CoinFlip, GamblersRuin, RandomWalk, BiasedBandit) follow the same proof pattern:

1. **One action dominates in expected immediate reward**: $\mathbb{E}[\text{head}(a)] < \mathbb{E}[\text{head}(b)]$
2. **Head preservation is trivial**: The ranking hypothesis directly implies $\text{head}(a) \leq_r \text{head}(b)$
3. **Tail preservation is vacuous**: The condition $\text{head}(a) \equiv \text{head}(b)$ is absurd (e.g., $0 \equiv 1$)

4. **Absurd pattern match completes the proof**: The empty pattern `()` satisfies the tail obligation

When does this pattern apply? Whenever the ranked-higher action has *strictly* higher expected immediate reward. This covers many practical scenarios: betting vs. staying (GamblersRuin), walking vs. staying (RandomWalk), better vs. worse arm (BiasedBandit). **When might it fail?** If two actions have equal expected immediate reward but differ in their tails. In such cases, the tail proof becomes substantive—but also more informative about the MDP's structure.

11.6 The Stochastic Coinductive Homomorphism

The preservation property uses lexicographic dominance:

```
-- Stochastic CoindHomo with lexicographic preservation
record StochasticCoindHomo
  (State Action : Set) (StreamR : Set)
  (expected-action-value : State → Action → Stream StreamR)
  (_≤ᵣ_ : StreamR → StreamR → Set) : Set₁ where
  field
    _≤ₐ_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤ₐ_ s a b →
                  _≤ₛ-lex_ _≤ᵣ_ (expected-action-value s a) (expected-action-value s b)
```

The full implementation is in `src/CSHRL/Core/Stochastic.agda`, with the stochastic Environment Class in `src/CSHRL/EnvironmentClass/StochasticFiniteMDP.agda`.

11.7 Relationship Between Orderings

The two orderings form a logical hierarchy:

- **Pointwise ⇒ Lexicographic**: If $x \leq_s y$ at every timestep, then certainly $x \leq_{lex} y$.
- **Lexicographic ⇏ Pointwise**: Lexicographic is more permissive—it accepts orderings where early rewards "break ties" without requiring tail dominance.
- **Deterministic case**: Both coincide, since trajectories are fixed and the comparison is deterministic.

Why lexicographic is correct for stochastic: Expected values collapse distributions to scalars. When $\mathbb{E}[\text{head}(a)] < \mathbb{E}[\text{head}(b)]$, action b is already established as better—demanding further tail proofs adds no information. Lexicographic captures exactly this semantics.

11.8 Subsumption of Classical RL

Does the stochastic extension still subsume classical RL's expected-return optimization? Yes. By linearity of expectation:

$$\mathbb{E}\left[\sum_{t=0}^{N-1} r_t\right] = \sum_{t=0}^{N-1} \mathbb{E}[r_t]$$

Comparing partial sums of expected rewards is equivalent to comparing expected partial sums—exactly what classical RL optimizes.

Theorem 3 (Stochastic Subsumption—see Appendix)**.** *If rankings satisfy pointwise expected stream dominance, then for any finite horizon N, the ranked-higher action has higher expected cumulative reward.*

Since pointwise implies lexicographic, any MDP satisfying our lexicographic preservation *and* pointwise dominance (which includes all four verified examples) also subsumes classical argmax. The Appendix provides the complete proof in `appendix/StochasticSubsumption.agda`.

11.9 Summary: Stochastic Extension

The stochastic extension provides a complete parallel to the deterministic framework:

- **Infrastructure:** Finite distribution monad (`Probability.Finite`), stochastic core (`Core/Stochastic`), environment class (`StochasticFiniteMDP`)
- **Key insight:** Lexicographic coinductive comparison is the correct notion for expected streams—more permissive than pointwise, but still coinductive
- **Verified examples:** CoinFlip, GamblersRuin, RandomWalk, BiasedBandit (all postulate-free, `--safe`)
- **Subsumption:** Classical argmax is subsumed for expected rewards (Appendix)
- **Learning:** Extends directly—see §12.4
- **Future direction:** First-Order Stochastic Dominance (FOSD) for risk-sensitive orderings (see Future Work)

The proof pattern is uniform: when one action has strictly higher expected immediate reward, the lexicographic tail obligation becomes vacuous (absurd pattern match). The current expected-value semantics suffices for classical RL subsumption; FOSD extends to risk-averse agents.

12 Learning as Symmetry Restoration

The Finder computes rankings at a fixed depth. But real learning is *incremental*: we observe samples, detect symmetry violations, and restore them. This is the **learning loop**.

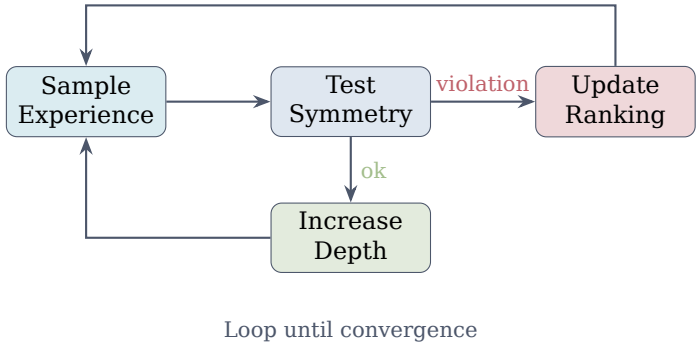

Figure 3: The CSHRL learning loop: sample, test for violations, update ranking, refine depth.

12.1 Violations and Swaps

A **violation** occurs when the ranking disagrees with observed traces:

- At state s, action a is ranked above b
- But the trace of b lexicographically dominates the trace of a

The fix is simple: **swap** a and b in the ranking. This directly corrects the mistake.

12.2 Monotonicity Guarantee

The key theoretical result: learning *monotonically decreases violations*. The full implementation is in `src/CSHRL/Learning/Base.agda`. Here we define and type-check the core concepts:

```
-- A Violation records where ranking disagrees with the oracle
record Violation : Set where
  constructor violation
  field
    viol-state  : State
    viol-better : Action -- Should be ranked higher
    viol-worse : Action -- Should be ranked lower
    viol-depth : ℕ      -- Depth at which detected

-- The dominance oracle (ground truth from traces at sufficient depth)
DominanceOracle : Set
DominanceOracle = State → Action → Action → Bool


-- Swap the violated pair in the ranking
swap-in-list : Action → Action → List Action → List Action
swap-in-list _ _ [] = []
swap-in-list better worse (x :: xs) with x ≟ₐ worse
... | yes _ = better :: worse :: remove-action better xs
... | no _ = x :: swap-in-list better worse xs

-- Global updater: swaps the pair in the violated state's ranking
global-swap-updater : Violation → ExplicitRanking → ExplicitRanking
global-swap-updater v ranking s =
  swap-in-list (Violation.viol-better v) (Violation.viol-worse v) (ranking s)
```

Theorem 4 (Swap Monotonicity)**.** *Let v be a violation with actions $(better, worse)$. After swapping:*

1. *The violated pair is now correctly ordered*
2. *Unrelated pairs are unchanged*
3. *Total violations decrease by at least 1*

The proof proceeds via three verified lemmas (full proofs in `src/CSHRL/Learning/Base.agda`):

```
-- LEMMA 1: Swap fixes the violated pair
SwapFixesPair : Set
SwapFixesPair = ∀ (better worse : Action) (xs : List Action) →
  (better ≡ worse → ⊥) →
  is-dominated-by (swap-in-list better worse xs) worse better ≡ true

-- LEMMA 2: Swap preserves unrelated pairs
SwapPreservesUnrelated : Set
SwapPreservesUnrelated = ∀ (better worse a b : Action) (xs : List Action) →
  (a ≡ better → ⊥) → (a ≡ worse → ⊥) →
  (b ≡ better → ⊥) → (b ≡ worse → ⊥) →
  is-dominated-by xs a b ≡ is-dominated-by (swap-in-list better worse xs) a b


-- LEMMA 3: Per-state violations decrease
ViolationsAtStateMono : DominanceOracle → Set
ViolationsAtStateMono oracle =
  ∀ (s : State) (v : Violation) (ranking : ExplicitRanking) →
```

```
    let swapped = global-swap-updater v ranking
    in count-violations-at swapped oracle s ≤ count-violations-at ranking oracle s

-- THEOREM: Total violations across all states decrease
TotalViolationsMono : DominanceOracle → List State → Set
TotalViolationsMono oracle all-states =
  ∀ (v : Violation) (ranking : ExplicitRanking) →
  oracle (Violation.viol-state v) (Violation.viol-better v)
         (Violation.viol-worse v) ≡ true →
  let swapped = global-swap-updater v ranking
  in count-total-violations swapped oracle all-states ≤
     count-total-violations ranking oracle all-states
```

This gives a convergence bound:

$$\text{max-violations} \leq |S| \times \binom{|A|}{2} = |S| \times \frac{|A|(|A|-1)}{2}$$

The convergence theorem and its full proof are in `src/CSHRL/Learning/Base.agda`:

```
-- CONVERGENCE THEOREM: Learning terminates with zero violations
ConvergenceTheorem : DominanceOracle → List State → Set
ConvergenceTheorem oracle all-states =
  StrictDecrease oracle all-states →
  ∀ (ranking₀ : ExplicitRanking)
    (find-viol : ExplicitRanking → Maybe Violation) →
  FindViolValid find-viol →
  (∀ r v → find-viol r ≡ just v →
     oracle (Violation.viol-state v) (Violation.viol-better v)
            (Violation.viol-worse v) ≡ true) →
  (∀ r → find-viol r ≡ nothing →
     count-total-violations r oracle all-states ≡ 0) →
  ∃[ n ] (n ≤ count-total-violations ranking₀ oracle all-states ×
          count-total-violations (iterated-update find-viol ranking₀ n)
            oracle all-states ≡ 0)
```

Learning is guaranteed to converge in at most `max-violations` swaps. The full proofs are in `CSHRL.Learning.Base`.

Remark 3 (Full-ranking learning dissolves the exploration/exploitation trade-off). The convergence bound $|S| \times \binom{|A|}{2}$ counts *all* pairwise violations, including those between the two worst actions. To reach zero violations the learner must correctly order every pair—not just identify the argmax. This has a structural consequence: uniform pairwise sampling is sufficient for convergence, and no separate exploration mechanism (ϵ-greedy, entropy bonus, UCB) is needed.

The reason is that correctly ranking any pair of actions requires understanding both of their infinite future trajectories, which in turn requires understanding the shared environment dynamics (transition function, reward structure). In a parametric representation such as a deep neural network, this effect is amplified: the parameters governing the prediction of low-value action rankings are the same parameters governing high-value predictions. A violation between two bad actions produces gradient signal that updates shared parameters globally, forcing the representation toward comprehensive environment understanding. This contrasts with scalar value functions, where approximation errors in low-value regions are invisible to argmax selection and therefore never corrected.

12.3 Practical Learning Interface

The learning infrastructure provides a *curried* (stateful) interface that supports practical deployment:

- **Checkpointing:** The `LearnerState` record captures training progress (current depth, samples seen, violations detected). Save it to pause training; resume by passing it to `train-step`.
- **Incremental learning:** Process samples one at a time with `train-step`, or in batches with `train-batch`. Inspect state between samples.
- **Training traces:** The `training-trace` function returns the full history of learner states—useful for plotting convergence curves or debugging.
- **Active refinement:** The `ActiveLearner` maintains an explicit ranking that is directly updated on violation (swap), rather than re-querying the Finder. This provides faster convergence in practice.
- **Policy materialization:** A `PolicyTable` is a concrete list of (state, action-ranking) pairs—the verified analogue of trained weights in a DNN. The function `materialize-on-path` builds such a table by following the optimal trajectory from a given initial state, calling `find-ranking` once per visited state; `materialize-at` builds one for an arbitrary list of states. At deployment time, `lookup` retrieves the cached ranking in $O(n)$; for states not in the table a fallback to the online Finder guarantees that no state is ever unhandled. All three EC learning modules (deterministic, combinatorial, stochastic) expose materialization.

These features are demonstrated in `src/CSHRL/Tasks/Verified/CurriedLearnerDemo.agda`, which uses a "Late Bloomer" MDP where traces are indistinguishable at low depth but diverge later—validating that depth increase and violation-driven swaps correctly discover the optimal ranking.

12.4 Stochastic Learning Extension

The learning infrastructure extends directly to stochastic MDPs. The only change: violation detection compares *expected traces* instead of deterministic traces.

```
-- Stochastic violation test: compare EXPECTED traces
stoch-test-pair : ℕ → StochSample → Maybe StochViolation
stoch-test-pair k (stoch-sample s a b) with finder-ranking k s a b |
                                    expected-trace-action s a k ≤ᵗᵇ expected-trace-action s b k
... | true | false = just (stoch-violation s b a k) -- Ranking disagrees
... | _    | _ = nothing -- No violation
```

All monotonicity guarantees carry over: swaps still strictly decrease violation count, convergence still holds. The full stochastic learning module is in `src/CSHRL/Learning/StochasticFiniteMDP.agda`.

12.5 Combinatorial Placement Learning

The learning infrastructure also extends to `CombinatorialPlacementMDP`. The module `CSHRL.Learning.CombinatorialPlacementMDP` reuses the trace and ranking infrastructure already defined in the environment class (short-circuit traces for absorbing Dead/Solved states, binary rewards, horizon-bounded search) and adds the learning-specific layer: violation detection, restricted (unavailability-aware) trace computation, the depth-increase learning loop, and both the curried `Learner` and `ActiveLearner` interfaces.

The key structural difference from the deterministic and stochastic learning modules is that absorbing states are short-circuited during trace computation: Dead states immediately produce the zero-trace $[0, 0, ...]$ and Solved states produce $[R, R, ...]$, avoiding the exponential blowup that would arise from expanding the full game tree. This makes violation detection practical even for problems like 8-Queens.

Together with the verification bridge (§10.6), this gives CombinatorialPlacementMDP tasks *both* a static guarantee (the CoindHomo from `WithTraceBridge`) *and* a dynamic discovery mechanism (the learning loop). The bridge certifies that the Finder's ranking is optimal; the learner discovers it from experience.

Validation: 8-Queens policy extraction. The module `Queens8Learning` (`src/CSHRL/Tasks/Verified/Queens8Learning.agda`) instantiates the combinatorial placement learning infrastructure for the full 8-Queens problem. Training calls `materialize-on-path` from the empty board, performing a single game-tree search per board position along the optimal trajectory and storing each result in a `PolicyTable`—eight entries total. The resulting table *is* the learned policy: a verified list of (state, action-ranking) pairs, the formal analogue of trained weights.

Deployment uses `run-policy`, which looks up each encountered state in the table. On a hit the action is read off instantly; on a miss (a state not visited during training) the function falls back to `find-policy`—the same verified Finder that built the table—so no state is ever unhandled. The main result is checked by `refl`:

```
test-solution : run-policy policy horizon (Ongoing []) horizon
              ≡ C0 :: C4 :: C7 :: C5 :: C2 :: C6 :: C1 :: C3 :: []
test-solution = refl
```

This confirms that the materialized policy, extracted entirely through the learning infrastructure, produces a valid non-attacking 8-Queens placement—verified end-to-end in Agda with `--safe` and zero postulates.

13 Instant Adaptation to Action Unavailability

In many practical settings—robotics, network routing, game playing—actions become unavailable at runtime (a joint locks, a link fails, a move is blocked). Classical RL stores only the argmax action and must recompute the policy from scratch at cost $O(|S| \cdot |A|)$ or worse.

Because CSHRL maintains a *total ranking* over all actions, adaptation is $O(1)$: when the top-ranked action becomes unavailable, the second-ranked action is already known and its optimality among the remaining actions is guaranteed by the restricted preservation theorem below.

13.1 Restricted Preservation

Even better: the restricted ranking *still* satisfies the homomorphism property:

Theorem 5 (Restricted Preservation)**.** *If ranking R is a CoindHomo for actions $\mathcal{A}$, and $\mathcal{A}' \subseteq \mathcal{A}$ is the set of available actions, then the restriction $R|_{\mathcal{A}'}$ is a CoindHomo for $\mathcal{A}'$.*

We formalize this with explicit types for availability and restricted rankings:

```
-- Availability predicate: which actions are currently executable
Available : Set
Available = Action → Bool

-- Filter a list to only available actions
filter-available : Available → List Action → List Action
filter-available _ [] = []
filter-available p (x :: xs) =
  if p x then x :: filter-available p xs else filter-available p xs

-- Restricted ranking: only rank available actions
restricted-explicit-ranking : Available → ExplicitRanking → ExplicitRanking
restricted-explicit-ranking avail ranking s = filter-available avail (ranking s)
```

The preservation property follows immediately: since preservation is pairwise, the fact that $a \preceq_a b$ implies `action-value` $s\ a \preceq_s$ `action-value` $s\ b$ does not depend on which other actions exist. Removing unavailable actions from the ranking leaves the relative order—and the action-values—of the remaining actions unchanged.

```agda
-- Filter by predicate (returns elements where p is true)
filter-bool : (Action → Bool) → List Action → List Action
filter-bool _ [] = []
filter-bool p (x :: xs) =
  if p x then x :: filter-bool p xs else filter-bool p xs

-- Demote unavailable actions to the bottom of the ranking
demote-unavailable : Available → List Action → List Action
demote-unavailable avail xs = available ++ unavailable
  where
    available = filter-bool avail xs
    unavailable = filter-bool (λ a → not (avail a)) xs


-- O(1) adaptation: demote a forbidden action to end of list
make-action-unavailable : Action → ExplicitRanking → ExplicitRanking
make-action-unavailable forbidden ranking s = demote-to-end forbidden (ranking s)
```

Classical RL stores only the argmax; when that action fails, the policy must be recomputed at cost $O(|S| \cdot |A|)$. CSHRL stores a total ranking, so when the top action fails the second-best is already available—$O(1)$ lookup.

14 Verified Instantiations: From Theory to Tasks

The examples in this section serve a specific purpose: they *validate* that the formal machinery instantiates correctly. Each example that type-checks is a proof that CSHRL applies to that task class. We present three instantiations of increasing complexity.

14.1 TwoState: Minimal Working Example

The simplest possible MDP: two states, two actions, one correct ranking.

```agda
module TwoStateExample where
  open import Data.Nat using (_≤_; z≤n; s≤s)
  open import Data.Nat.Properties using (≤-refl)

  data TwoState : Set where
    Start : TwoState
    Goal : TwoState

  data TwoAction : Set where
    Go : TwoAction
    Stay : TwoAction

  two-step : TwoState → TwoAction → TwoState × ℕ
  two-step Start Go = (Goal , 1)
  two-step Start Stay = (Start , 0)
  two-step Goal _    = (Goal , 1) -- Absorbing

  two-actions : List TwoAction
  two-actions = Go :: Stay :: []

  open Core TwoState TwoAction ℕ two-step _≤_ _⊔_ 0 two-actions
```

This validates that the Core module instantiates correctly with natural number rewards. At `Start`, `Go`'s future is $(1, 1, 1, ...)$ and `Stay`'s is $(0, 0, 0, ...)$; the ranking `[Go, Stay]` satisfies the coinductive homomorphism at every step. The full preservation proof is in `src/CSHRL/Tasks/Verified/TwoState.agda`, with no postulates.

14.2 Delayed Gratification: Depth-Discovery Phenomenon

This example demonstrates how the Finder discovers structure through depth increase when rewards are hidden deep in the future.

```
data DGState : Set where
  Start : DGState
  PathA : DGState -- Immediate but dead-end
  PathB : DGState -- Delayed but rewarding
  End : DGState

data DGAction : Set where
  GoA : DGAction -- Quick path (0 forever)
  GoB : DGAction -- Delayed path (0 → 1)
  Wait : DGAction -- Idle

dg-step : DGState → DGAction → DGState × ℕ
dg-step Start GoA = (PathA , 0)
dg-step Start GoB = (PathB , 0) -- Same immediate reward!
dg-step PathA _ = (PathA , 0)   -- Stuck at 0
dg-step PathB _ = (End , 1)     -- Reveals hidden reward
dg-step End _   = (End , 1)     -- Absorbing at 1
dg-step _   _   = (End , 0)     -- Catch-all (e.g., Start + Wait)

all-actions : List DGAction
all-actions = GoA :: GoB :: []

-- Instantiate Core with this environment
open Core DGState DGAction ℕ dg-step _≤_ _⊔_ 0 all-actions
```

The key insight: at depth 1, both actions yield trace $[0]$—indistinguishable. At depth 2 the traces diverge: `GoA` $\to [0,0]$, `GoB` $\to [0,1]$. Lexicographic comparison reveals the winner. Full implementation in `src/CSHRL/Tasks/Classic/DelayedGratification.agda`.

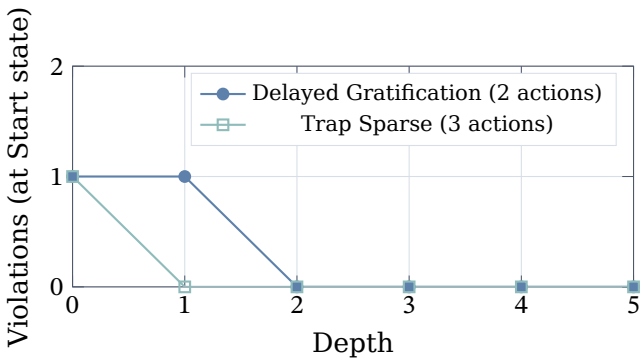

Figure 4: Violations at Start state drop to zero as depth increases. Delayed Gratification requires depth 2 to distinguish equal-immediate-reward paths; Trap Sparse resolves at depth 1. Both are verified in `src/CSHRL/Tasks/Classic/`.

The Finder's lexicographic comparison thus discovers temporal structure that scalar methods would require value propagation to find.

14.3 Key-Door-Treasure: State-Dependent Rankings

A robot must collect a key, unlock a door, then reach treasure. This demonstrates *state-dependent* rankings and the "ranking flip" phenomenon.

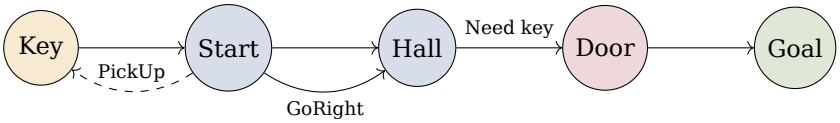

Figure 5: Key-Door-Treasure: the optimal action depends on whether the key is held.

The key phenomenon is the *ranking flip*:

State	Without Key	With Key
Start	[PickUp, GoRight, GoLeft]	[GoRight, GoLeft, PickUp]
Hall	[GoLeft, GoRight, Wait]	[GoRight, Wait, GoLeft]

The moment the key is acquired, the ranking inverts—a phase transition encoded directly in the traces rather than via gradual value propagation:

- Without key: `GoRight` $\to [0, 0, 0, \ldots]$ (blocked)
- With key: `GoRight` $\to [0, 0, 100, 100, \ldots]$ (treasure)

This validates that state-dependent rankings instantiate correctly under the `FiniteDeterministicMDP` environment class. The full implementation is in `src/CSHRL/Tasks/Classic/KeyDoor.agda`.

15 On Undecidability

A natural objection: the coinductive symmetric homomorphism is undecidable in general—verifying an infinite tower of conditions requires checking infinitely many timesteps.

We view this as analogous to AIXI [4]: an uncomputable ideal that nonetheless organizes a hierarchy of practical approximations:

- Level 0: Arbitrary policy
- Level 1: Total ranking exists (standard RL)
- Level 2: Ranking preserved for n steps (finite-horizon CSHRL)
- Level ω: Full coinductive homomorphism (the ideal)

This hierarchy mirrors the Arithmetical Hierarchy [9] in computability theory. Crucially, Level ω is *achievable* for structured domains: the 8-Queens instance (§10.6) attains a full `CoindHomo`—Level ω—by exploiting the binary reward structure and finite game depth of placement problems. The undecidability of the general case does not preclude decidable sub-classes; environment classes serve precisely to identify and mechanise these sub-classes.

16 Future Work

16.1 Beyond the Current Stochastic Extension

The stochastic framework (§11) currently handles finite discrete distributions with expected-value comparison. Natural extensions include:

- **Continuous distributions:** Extend from `List (A × ℕ)` to measure-theoretic distributions.
- **Partially observable MDPs:** Belief-state distributions with stochastic transitions.

16.1.1 Risk-Sensitive Extension: The FOSD Isomorphism Conjecture

The most intriguing extension is from expected values to **First-Order Stochastic Dominance (FOSD)**—a risk-sensitive ordering on distributions. FOSD-dominance ($\mu \geq_{\text{FOSD}} \nu$) means the CDF of μ lies everywhere below ν's:

$$\forall r.\, P_\mu(X \leq r) \leq P_\nu(X \leq r)$$

This is strictly stronger than expected-value comparison: $\mu \geq_{\text{FOSD}} \nu \Rightarrow \mathbb{E}[\mu] \geq \mathbb{E}[\nu]$, but not vice versa. An action FOSD-dominating another is preferred by *all* risk-averse decision makers, not just expected-value maximizers.

The Conjecture: Define coinductive FOSD (comparing marginal distributions at each step) and pointwise FOSD (comparing at all timesteps). The Appendix conjectures these are isomorphic—extending the Stream Isomorphism (§7) to distributions.

Current infrastructure provides the Giry monad foundation (`Probability.Finite`). Proving the conjecture requires additional machinery: CDF computation for `Dist Reward`, FOSD comparison, and conditional tail distributions preserving correlation structure. See the Appendix for full details.

16.2 Additional Extensions

The modular Environment Class design invites extensions:

- **Continuous State/Action Spaces:** Parameterize by metric spaces with approximation guarantees.
- **Partially Observable MDPs:** Lift rankings to belief states.
- **Multi-Agent Settings:** Rankings over joint action spaces with game-theoretic equilibrium conditions.
- **Quantum violation detection:** CSHRL's permutation structure over rankings may admit Grover-style speedup for violation search [7, 8], though formalizing this connection remains open.
- **Program synthesis as policy representation:** The full-ranking objective (Remark 3) naturally extends to synthesized programs: a program that satisfies all pairwise ranking constraints on observed states must encode sufficient environment structure to generalize to unseen states—analogous to how a DNN trained on violation-correction must develop shared representations. Exploring this connection to inductive program synthesis is a promising direction.

17 Conclusion

We have presented Coinductive Symmetric Homomorphism Reinforcement Learning (CSHRL), a framework that redefines optimality as *structure preservation* rather than scalar maximization. A ranking is optimal when it forms a coinductive homomorphism into the ordering of infinite reward streams—dominance at every timestep, not just in aggregate.

Five main results, all machine-verified in Agda, support this framework:

1. **Subsumption:** The coinductive property implies classical argmax optimality for any finite horizon.
2. **Monotonic learning:** Violation-correction converges in a bounded number of swaps.
3. **Instant adaptation:** Action unavailability requires $O(1)$ lookup, not full recomputation.
4. **Stochastic extension:** The framework lifts to probabilistic environments via the Giry monad, where lexicographic coinductive comparison provides the appropriate ordering on expected reward streams.

5. **Scalable EC-based verification:** A modular Environment Class architecture provides automatic trace-to-stream bridges; the 8-Queens instance achieves a full `CoindHomo` for 8^8 candidate placements with zero postulates.

The stochastic extension resolves the gap between trace-level and stream-level dominance, yielding fully verified stochastic examples (CoinFlip, GamblersRuin, RandomWalk, BiasedBandit) with no postulates. The placement EC result demonstrates that the modular architecture scales to non-trivial problem sizes while preserving the strongest correctness guarantee. On the practical side, the learning infrastructure supports *policy materialization*: learned rankings are stored as concrete tables and deployed via lookup rather than recomputation, closing the gap between verified theory and the train-then-deploy paradigm of practical RL systems. This pipeline is validated end-to-end on 8-Queens, where a materialized policy table produces a verified non-attacking placement.

18 Reproducibility and Build

The paper is a literate Agda document. Building the PDF type-checks all code.

- **Tested toolchain:** Agda 2.7.x (2.7.0 recommended), Agda StdLib 2.0, LuaLaTeX.
- **Build commands:** In `literate/`, run `make all` (or `agda --latex CSHRL.lagda.tex` followed by `lualatex` on the generated `CSHRL.tex`).
- **Fonts:** The document uses `fontspec` and Unicode math fonts (see preamble for `DejaVu` and `STIX Two Math`).
- **Artifacts:** The final PDF is copied to `literate/CSHRL.pdf`.

References

[1] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.

[2] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.

[3] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of ICML*, pp. 1861–1870.

[4] Hutter, M. (2005). *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer.

[5] Rutten, J. J. M. M. (2000). Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80.

[6] van der Pol, E., Worrall, D., van Hoof, H., Oliehoek, F., and Welling, M. (2020). MDP homomorphic networks: Group symmetries in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[7] Mutschler, M., Schön, C., and Doernbach, T. (2022). A survey on quantum reinforcement learning. *arXiv preprint arXiv:2211.03464*.

[8] Sannai, A., Imaizumi, M., and Kawano, M. (2024). An introduction to quantum reinforcement learning. *arXiv preprint arXiv:2409.05846*.

[9] Rogers, H. (1987). *Theory of Recursive Functions and Effective Computability*. MIT Press.

[10] Giry, M. (1982). A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, Lecture Notes in Mathematics, vol. 915, pp. 68–85. Springer.

[11] Skalse, J., Abate, A., and Hammond, L. (2022). Lexicographic multi-objective reinforcement learning. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 3430–3436.

[12] Tercan, H. and Prabhu, S. (2024). Thresholded lexicographic ordered multiobjective reinforcement learning. In *European Conference on Artificial Intelligence (ECAI)*.

[13] García, J. and Fernández, F. (2015). A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(1):1437–1480.

[14] Wirth, C., Akrour, R., Neumann, G., and Fürnkranz, J. (2017). A survey of preference-based reinforcement learning methods. *Journal of Machine Learning Research*, 18(136):1–46.

[15] Zap, A. and Joppen, T. (2019). Deep ordinal reinforcement learning. *arXiv preprint arXiv:1905.05344*.

[16] Anonymous (2025). Learning ordinal probabilistic reward from preferences. *OpenReview preprint*.

[17] Lesage, B. and Ong, C. H. L. (2020). CertRL: Formalizing convergence proofs for value and policy iteration in Coq. *arXiv preprint arXiv:2009.11403*.

[18] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*.